

Министерство образования Республики Беларусь  
Учреждение образования  
«Брестский государственный технический университет»  
Кафедра ИИТ

Лабораторная работа № 1  
По дисциплине « Обработка изображений в интеллектуальных системах »  
Тема: «Обучение классификаторов средствами библиотеки PyTorch»

Выполнил:  
Студент 4 курса  
Группы ИИ-26  
Рудь В.В.  
Проверил:  
Андренко К.В.

## Вариант 15

**Цель:** научиться конструировать нейросетевые классификаторы и выполнять их обучение на известных выборках компьютерного зрения

### Задачи лабораторной работы

1. Выполнить конструирование своей модели СНС, обучить ее на выборке по заданию (использовать `torchvision.datasets`). Предпочтение отдавать как можно более простым архитектурам, базирующимся на базовых типах слоев (сверточный, полносвязный, подвыборочный, слой нелинейного преобразования). Оценить эффективность обучения на тестовой выборке, построить график изменения ошибки (`matplotlib`);
2. Ознакомьтесь с state-of-the-art результатами для предлагаемых выборок (из материалов в сети Интернет). Сделать выводы о результатах обучения СНС из п. 1;
3. Реализовать визуализацию работы СНС из пункта 1 (выбор и подачу на архитектуру произвольного изображения с выводом результата);
4. Оформить отчет по выполненной работе, загрузить исходный код и отчет в соответствующий репозиторий на github.

|    |                               |       |           |
|----|-------------------------------|-------|-----------|
| 15 | STL-10<br>(размеченная часть) | 96X96 | Adadelata |
|----|-------------------------------|-------|-----------|

Код программы

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torch.utils.data import DataLoader
import torchvision
import torchvision.transforms as transforms
import matplotlib.pyplot as plt
import numpy as np
import random

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
batch_size = 128
num_epochs = 25
lr = 1.0 # Adadelata использует высокий lr
num_classes = 10
```

```
stl10_classes = [
    "airplane", "bird", "car", "cat", "deer",
    "dog", "horse", "monkey", "ship", "truck"
]
```

```
train_transform = transforms.Compose([
    transforms.RandomCrop(96, padding=4),
    transforms.RandomHorizontalFlip(),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.4469, 0.4399, 0.4066],
                          std=[0.2603, 0.2566, 0.2713])
])
```

```
test_transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.4469, 0.4399, 0.4066],
                        std=[0.2603, 0.2566, 0.2713])
])
```

```
train_dataset = torchvision.datasets.STL10(
    root="./data",
    split="train",
    download=True,
    transform=train_transform
)
```

```
test_dataset = torchvision.datasets.STL10(
    root="./data",
    split="test",
    download=True,
    transform=test_transform
)
```

```
# ВАЖНО ДЛЯ WINDOWS → num_workers=0
train_loader = DataLoader(train_dataset, batch_size=batch_size,
                          shuffle=True, num_workers=0)
test_loader = DataLoader(test_dataset, batch_size=batch_size,
                        shuffle=False, num_workers=0)
```

```
class SimpleSTL10CNN(nn.Module):
    def __init__(self):
        super(SimpleSTL10CNN, self).__init__()
```

```
self.conv1 = nn.Conv2d(3, 32, kernel_size=3, padding=1)
self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
self.pool1 = nn.MaxPool2d(2, 2) #  $64 \times 48 \times 48$ 
```

```
self.conv3 = nn.Conv2d(64, 128, kernel_size=3, padding=1)
self.conv4 = nn.Conv2d(128, 256, kernel_size=3, padding=1)
self.pool2 = nn.MaxPool2d(2, 2) #  $256 \times 24 \times 24$ 
```

```
self.conv5 = nn.Conv2d(256, 256, kernel_size=3, padding=1)
self.pool3 = nn.MaxPool2d(2, 2) #  $256 \times 12 \times 12$ 
```

```
self.fc1 = nn.Linear(256 * 12 * 12, 1024)
self.fc2 = nn.Linear(1024, num_classes)
self.dropout = nn.Dropout(0.5)
```

```
def forward(self, x):
    x = F.relu(self.conv1(x))
    x = F.relu(self.conv2(x))
    x = self.pool1(x)

    x = F.relu(self.conv3(x))
    x = F.relu(self.conv4(x))
    x = self.pool2(x)

    x = F.relu(self.conv5(x))
    x = self.pool3(x)

    x = x.view(x.size(0), -1)
    x = self.dropout(F.relu(self.fc1(x)))
    x = self.fc2(x)
    return x
```

```
model = SimpleSTL10CNN().to(device)
```

```
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adadelta(model.parameters(), lr=lr)
```

```
def evaluate():
    model.eval()
    total = 0
```

```
correct = 0
loss_sum = 0

with torch.no_grad():
    for images, labels in test_loader:
        images = images.to(device)
        labels = labels.to(device)

        outputs = model(images)
        loss = criterion(outputs, labels)

        loss_sum += loss.item() * images.size(0)
        _, predicted = torch.max(outputs, 1)

        total += labels.size(0)
        correct += (predicted == labels).sum().item()

return loss_sum / total, 100 * correct / total
```

```
train_losses = []
test_losses = []
test_accs = []

for epoch in range(num_epochs):
    model.train()
    running_loss = 0

    for images, labels in train_loader:
        images = images.to(device)
        labels = labels.to(device)

        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        running_loss += loss.item() * images.size(0)

    train_loss = running_loss / len(train_dataset)
    test_loss, test_acc = evaluate()
```

```

train_losses.append(train_loss)
test_losses.append(test_loss)
test_accs.append(test_acc)

print(f'Epoch {epoch+1}/{num_epochs} | "
      f"Train Loss: {train_loss:.4f} | "
      f"Test Loss: {test_loss:.4f} | "
      f"Test Acc: {test_acc:.2f}%")

plt.figure(figsize=(10, 4))

plt.subplot(1, 2, 1)
plt.plot(train_losses, label="Train Loss")
plt.plot(test_losses, label="Test Loss")
plt.legend()
plt.title("Loss")

plt.subplot(1, 2, 2)
plt.plot(test_accs, label="Test Accuracy")
plt.legend()
plt.title("Accuracy")

plt.show()

def denormalize(img):
    mean = np.array([0.4469, 0.4399, 0.4066])
    std = np.array([0.2603, 0.2566, 0.2713])
    img = img.numpy().transpose((1, 2, 0))
    img = std * img + mean
    return np.clip(img, 0, 1)

model.eval()
idx = random.randint(0, len(test_dataset) - 1)
image, label = test_dataset[idx]

with torch.no_grad():
    output = model(image.unsqueeze(0).to(device))
    _, pred = torch.max(output, 1)

true_class = stl10_classes[label]
pred_class = stl10_classes[pred.item()]

```

```
print(f"True: {true_class}, Predicted: {pred_class}")
```

```
plt.imshow(denormalize(image))  
plt.title(f"True: {true_class}\nPred: {pred_class}")  
plt.axis("off")  
plt.show()
```

## Результат

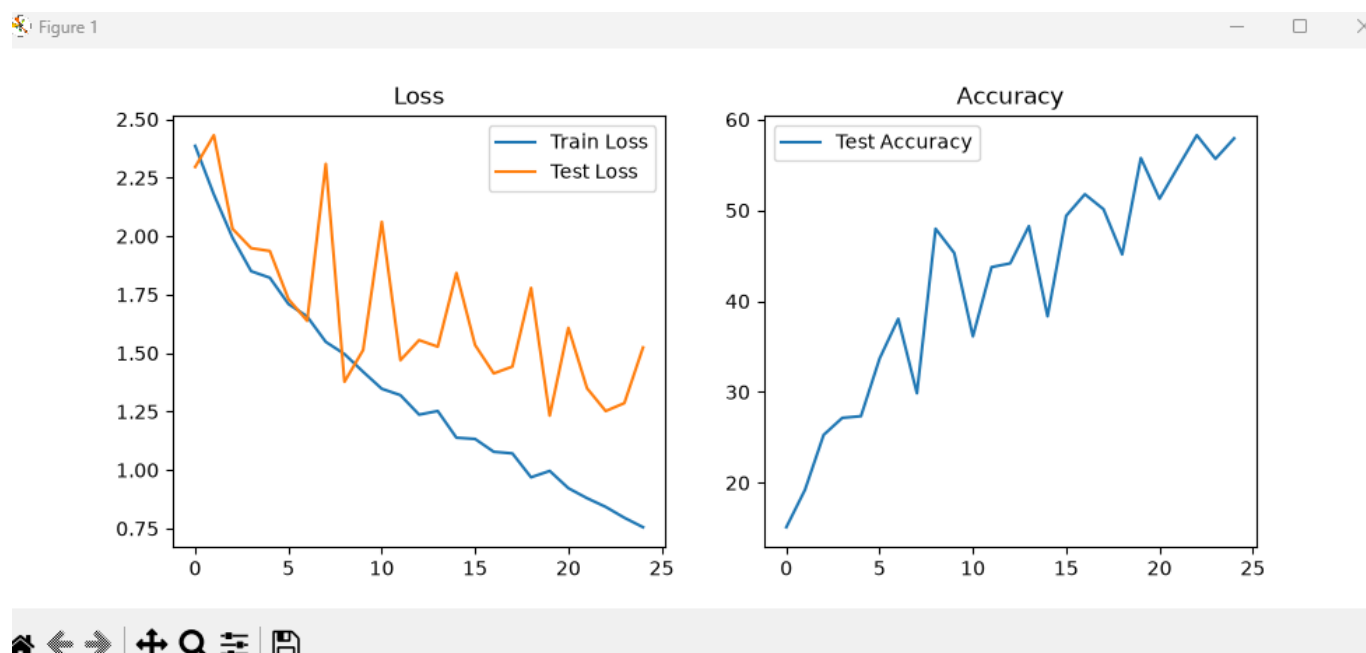


Figure 1

True: deer  
Pred: deer



(x, y) = (26.2, 33.6)  
[0.0941, 0.106, 0.051]

```
Epoch 1/25 | Train Loss: 2.3860 | Test Loss: 2.2962 | Test Acc: 15.11%
Epoch 2/25 | Train Loss: 2.1786 | Test Loss: 2.4319 | Test Acc: 19.23%
Epoch 3/25 | Train Loss: 1.9933 | Test Loss: 2.0317 | Test Acc: 25.27%
Epoch 4/25 | Train Loss: 1.8502 | Test Loss: 1.9486 | Test Acc: 27.15%
Epoch 5/25 | Train Loss: 1.8223 | Test Loss: 1.9366 | Test Acc: 27.34%
Epoch 6/25 | Train Loss: 1.7097 | Test Loss: 1.7305 | Test Acc: 33.69%
Epoch 7/25 | Train Loss: 1.6565 | Test Loss: 1.6381 | Test Acc: 38.06%
Epoch 8/25 | Train Loss: 1.5479 | Test Loss: 2.3093 | Test Acc: 29.86%
Epoch 9/25 | Train Loss: 1.4959 | Test Loss: 1.3772 | Test Acc: 47.98%
Epoch 10/25 | Train Loss: 1.4200 | Test Loss: 1.5131 | Test Acc: 45.30%
Epoch 11/25 | Train Loss: 1.3474 | Test Loss: 2.0613 | Test Acc: 36.12%
Epoch 12/25 | Train Loss: 1.3200 | Test Loss: 1.4695 | Test Acc: 43.75%
Epoch 13/25 | Train Loss: 1.2368 | Test Loss: 1.5555 | Test Acc: 44.16%
Epoch 14/25 | Train Loss: 1.2522 | Test Loss: 1.5274 | Test Acc: 48.26%
Epoch 15/25 | Train Loss: 1.1381 | Test Loss: 1.8430 | Test Acc: 38.34%
Epoch 16/25 | Train Loss: 1.1328 | Test Loss: 1.5345 | Test Acc: 49.40%
Epoch 17/25 | Train Loss: 1.0779 | Test Loss: 1.4131 | Test Acc: 51.77%
Epoch 18/25 | Train Loss: 1.0710 | Test Loss: 1.4423 | Test Acc: 50.12%
Epoch 19/25 | Train Loss: 0.9688 | Test Loss: 1.7788 | Test Acc: 45.15%
Epoch 20/25 | Train Loss: 0.9960 | Test Loss: 1.2324 | Test Acc: 55.76%
Epoch 21/25 | Train Loss: 0.9222 | Test Loss: 1.6081 | Test Acc: 51.27%
Epoch 22/25 | Train Loss: 0.8795 | Test Loss: 1.3496 | Test Acc: 54.80%
Epoch 23/25 | Train Loss: 0.8422 | Test Loss: 1.2521 | Test Acc: 58.27%
Epoch 24/25 | Train Loss: 0.7958 | Test Loss: 1.2860 | Test Acc: 55.67%
Epoch 25/25 | Train Loss: 0.7554 | Test Loss: 1.5239 | Test Acc: 57.92%
True: deer, Predicted: deer
```

Process finished with exit code 0

Простая система информационного поиска, основанная на базовой индексации и логических операторах, действительно способна находить документы по ключевым словам — и делает это достаточно уверенно, но у неё есть очевидные ограничения. Она легко путает документы, в которых встречаются одинаковые термины, и хуже справляется с запросами, которые требуют более точного ранжирования или понимания контекста. Чтобы приблизиться к качеству современных поисковых систем, недостаточно просто искать совпадения слов — нужны более продвинутые методы: взвешивание терминов через TF-IDF, использование косинусной меры сходства, расширение коллекции документов и применение алгоритмов ранжирования вроде BM25. Эти усложнения позволяют системе лучше различать похожие документы, устойчивее работать с разными типами запросов и выдавать результаты, которые действительно соответствуют ожиданиям пользователя.

**Вывод :** Прodelав данную лабораторную работу я научился конструировать нейросетевые классификаторы и выполнять из обучение на известных выборках компьютерного зрения.